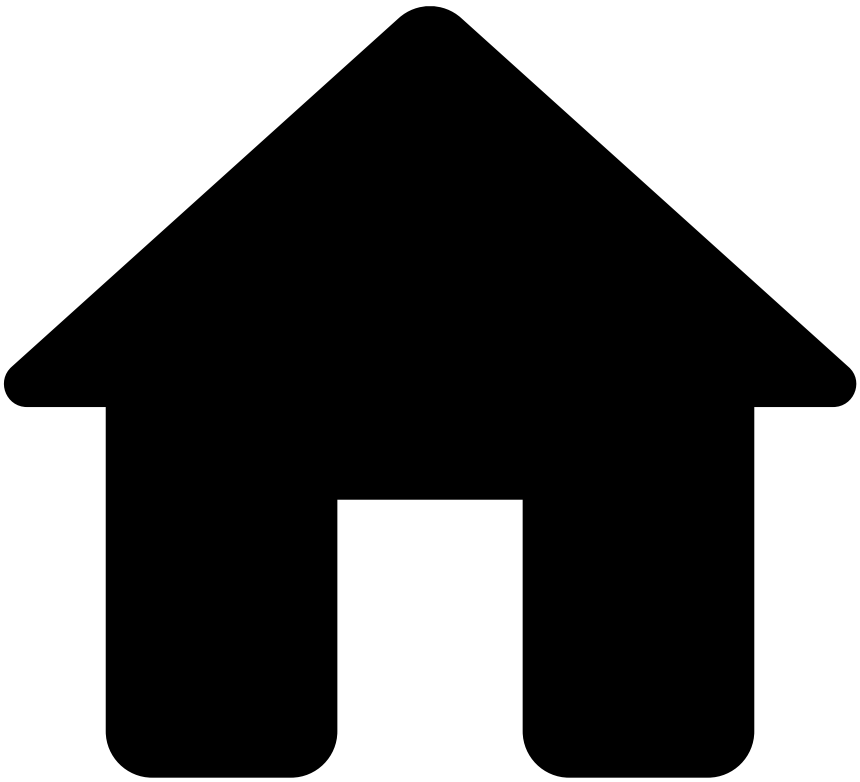


Data Types

Each field of a data source is characterized by a type that indicates its purpose in a data science context. Data sources are specified by their schema, and by the mapping between field types for data science and an underlying native data type. Possible field types are the following:

Simple fieldsâ□□

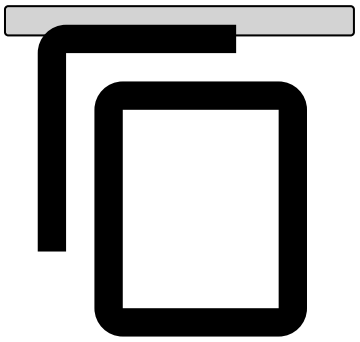
Type	Description	Configuration and native type	Example
Categorical			

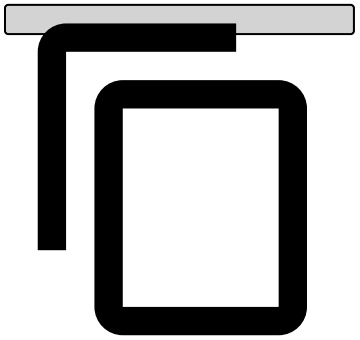
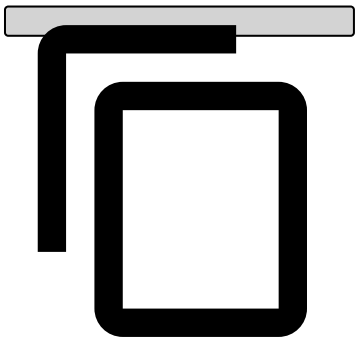
Type	Description	Configuration and native type	Example
	Fields that represent a finite amount of values of a discrete variable, that are useful in a Machine Learning setting. In Pulse you can use categorical fields to group specific business terms, or contract/domain specifications. As a heuristic, it is recommended that whenever the number of possible values exceeds ~10, you set the field type as String. In these cases, the number of possible values is too large for both a human and a machine learning algorithm to analyze and extract valuable insight. However, you can use Plans to transform (categorical or string) fields with too many categories into useful features. For example, using a UDF to categorize Merchant Category Code (MCC) values into Merchant Category Groups (MCG).	Each discrete value to be considered as valid for the field. All possible values of the field must have the same native type. Possible native types are: • String • Integer • Long • Boolean	CountryCode: 484, 124, 136, 840
Numeric	Fields that represent a continuous value. Pulse is able to generate arithmetic-based calculations and charts to analyze numeric fields such as distribution histograms, maximum, minimum, count, mean and standard deviation. You cannot perform Group By aggregations using Numeric fields as group by fields.	Possible native types are: • Integer • Float • Double	TransactionAmount
String	Text-based fields that in their raw format, usually, have no meaning in a data science context, when training a model. Although strings are not valuable in their raw format, they can be used for data science after some kind of processing or transformation with plans. For example, checking if an address starts with numbers or is in all caps. However, they may be useful in rules, for example to check if a value is in a list and may or may not be a categorical. Additionally, strings are also useful as support information for scored instances when visualizing model and rule evaluations, for example the client id, or even to analyse the performance of your models and rules broken down by these string fields.	Possible native types are: • String • Utf-8 String	CardholderAddress
Date Time	Fields that represent a date. Information like a timestamp is also numeric, however, looking at a timestamp and not considering its temporal properties would render it useless. Thus, Pulse processes data representations separately. Time-based information is useful for Pulse to validate data sequence and also very important to allow and facilitate time-aware operations. For example, helping the user selecting fields that represent time for time window-based aggregations.	Timestamp: A UNIX timestamp, in seconds or miliseconds. Date Time: A moment specified in a given format, for example dd/MM/yyyy.	Timestamp: 1350776834512 Date Time: 30/06/2016

Complex fieldsâ□□

Complex fields are used to describe structured data. When a complex field is defined it will represent a data structure like a list, map, tuple or set.

Type	Description	Configuration	Example

Map	Consider a transaction processing system with multiple POS. Each POS might have a lot of characteristics (e.g., battery usage, exterior temperature, etc), which might vary from one manufacturer to another. This information is difficult to maintain in a fixed data schema. Thus, you can define a map for POS properties (the information stored in the map must all have the same data type).
	
	Pulse only supports maps with primitive types as keys, and whose values are either primitive types or tuples.
		Types of key-value mapping. For example, Map of Integer - String	`test`
			`{`
			`  1: "Alaska",`
			`  2: "Minnesota"`
			`}`
			
			 

List	Consider a transaction that has two processing codes. While the data schema might have two different fields for each of the processing codes, it might be more convenient to include a list or a set type field containing both.	Type of all items in the list. For example, List of Integers.	`[1, 2, 44, 1]` 
Set	Keep in mind that the difference between sets and lists is that Sets don't allow duplicate values. Pulse only supports lists and sets whose elements are either primitive types or tuples.	Type of all items in the set. For example, Set of Strings.	`["Alaska", "Arizona"]` 

Tuple	Aggregate several items which can be of any simple type. Use tuples for the fields that might contain different known attributes.
	Pulse only supports tuples whose elements are primitive types.
		Types of each element of the tuple. For example, Tuple of 3 items.	`{`
			`  amount: 2,`
			`  country: "New York",`
			`  timestamp: 123456`
			`}`

Type	Description	Configuration	Example

Importing complex fields in CSV filesâ□□

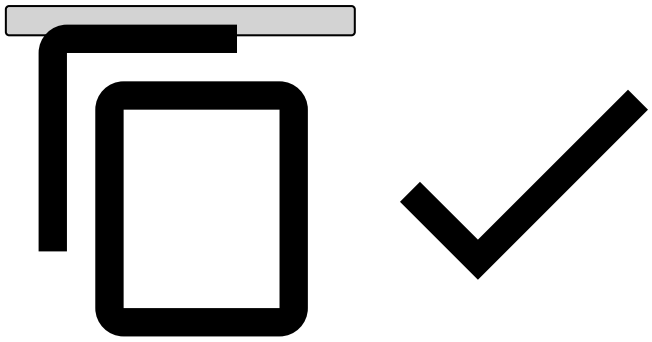
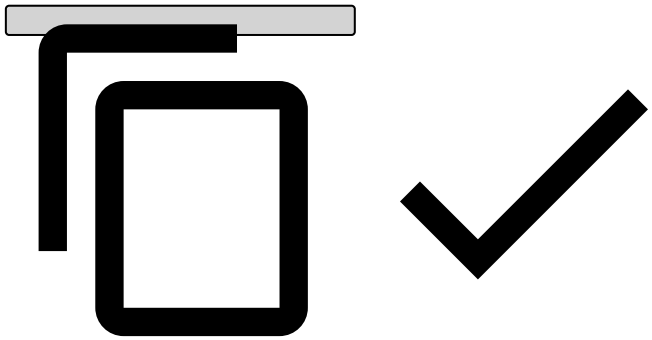
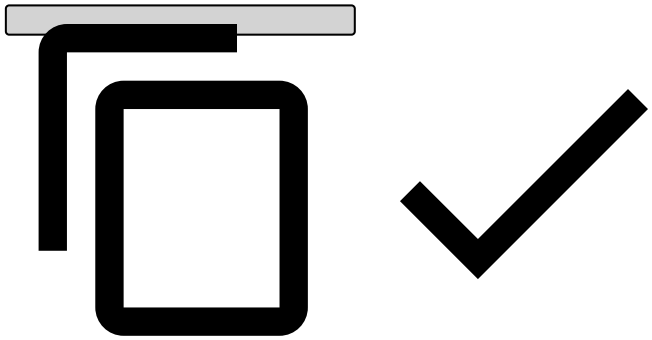
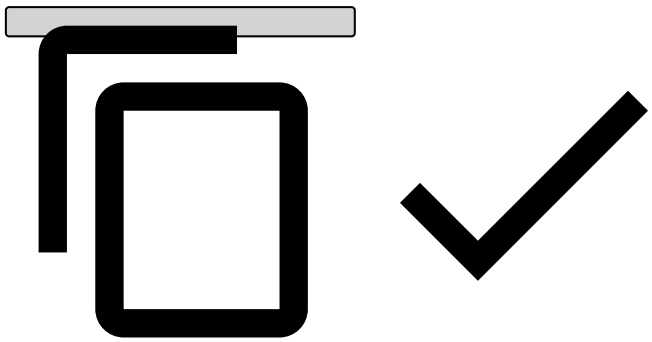
Internally in the Event Storage, Pulse uses system types directly. When importing data from a CSV file, fields with complex data must be represented as JSON strings.

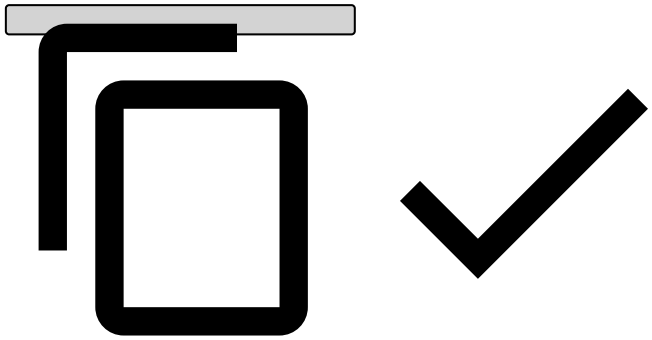
The syntax rules for complex data fields in CSV files are the following:

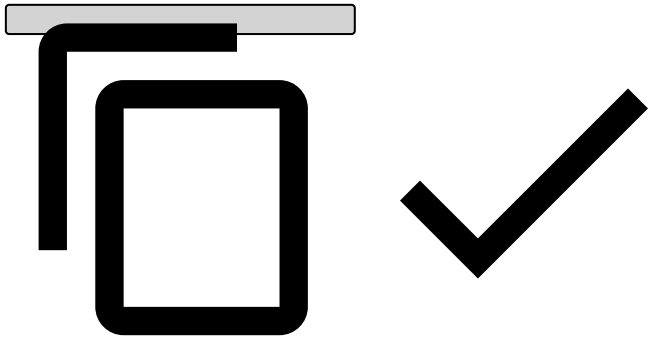
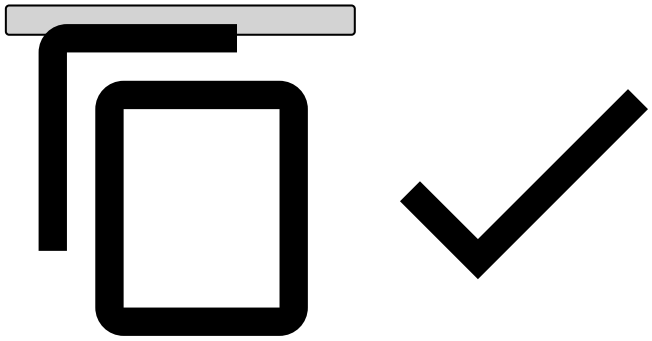
- Complex fields need to be represented as strings.
- Each complex field needs to be enclosed between quotation marks.
- If a complex field contains strings, the quotation marks around the strings must be escaped with a second quotation mark.
- Keys of complex fields must be strings. If in the schema they are defined as numeric type, they will be converted to the numeric type automatically, after reading the value from the string form in the CSV file.
- If you have stand-alone quotation marks within a string, it becomes \"".

As an example, let's take the following data row (displayed as a column for better readability):

Timestamp	`1350776834512`
Date	`30/06/2016`

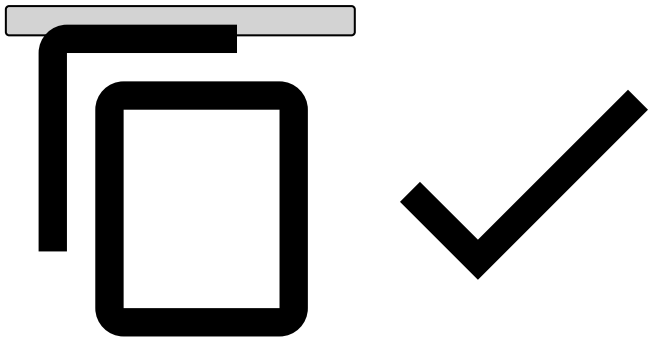
	
Country Code	`124` 
Transaction Amount	`124.99` 
Cardholder Address (string)	`"221b Baker St, London NW1 6XE"` 
POSProperties (tuple)	

	{
	  1: "Alaska",
	  2: "Minnesota"
	}
	 

ProcessingIDs (set of Integers)	[1, 2, 44, 1] 
ProcessingNames (list of strings)	["Alaska", "Arizona"] 

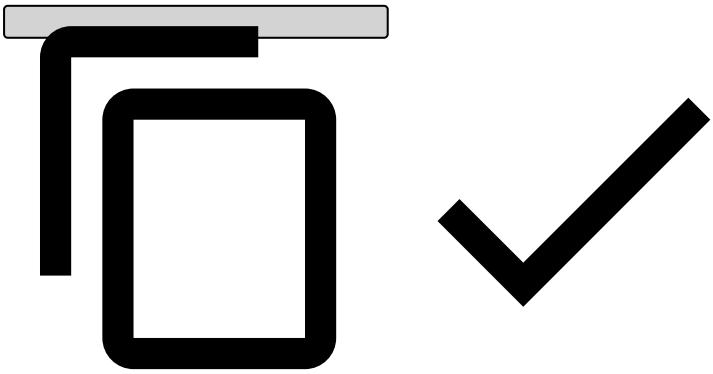
Items (list of tuples)	[
	  {
	  AmountUSD: 2.00,
	  Product: "Awesome Shirt"
	  },
	  {
	  AmountUSD: 242.00,
	  Product: "LED television 42" smartTV"
	  }

```
]
```



To import this row, the CSV file must look the following way:

```
Timestamp,Date,CountryCode,TransactionAmount,CardholderAddress,POSProperties,ProcessingIDs,Proce
ssingNames,Items
1350776834512,30/06/2016,124,124.99,"221b Baker St, London NW1 6XE","{""1"": ""Alaska"", ""2"":
""Minnesota""}","[1, 2, 44, 1]","[""Alaska"", ""Arizona""]","[{""AmountUSD"": 2.00, ""Product"":
""Awesome Shirt""}, {""AmountUSD"": 242.00, ""Product"": ""LED television 42\"" smartTV""}]"
```

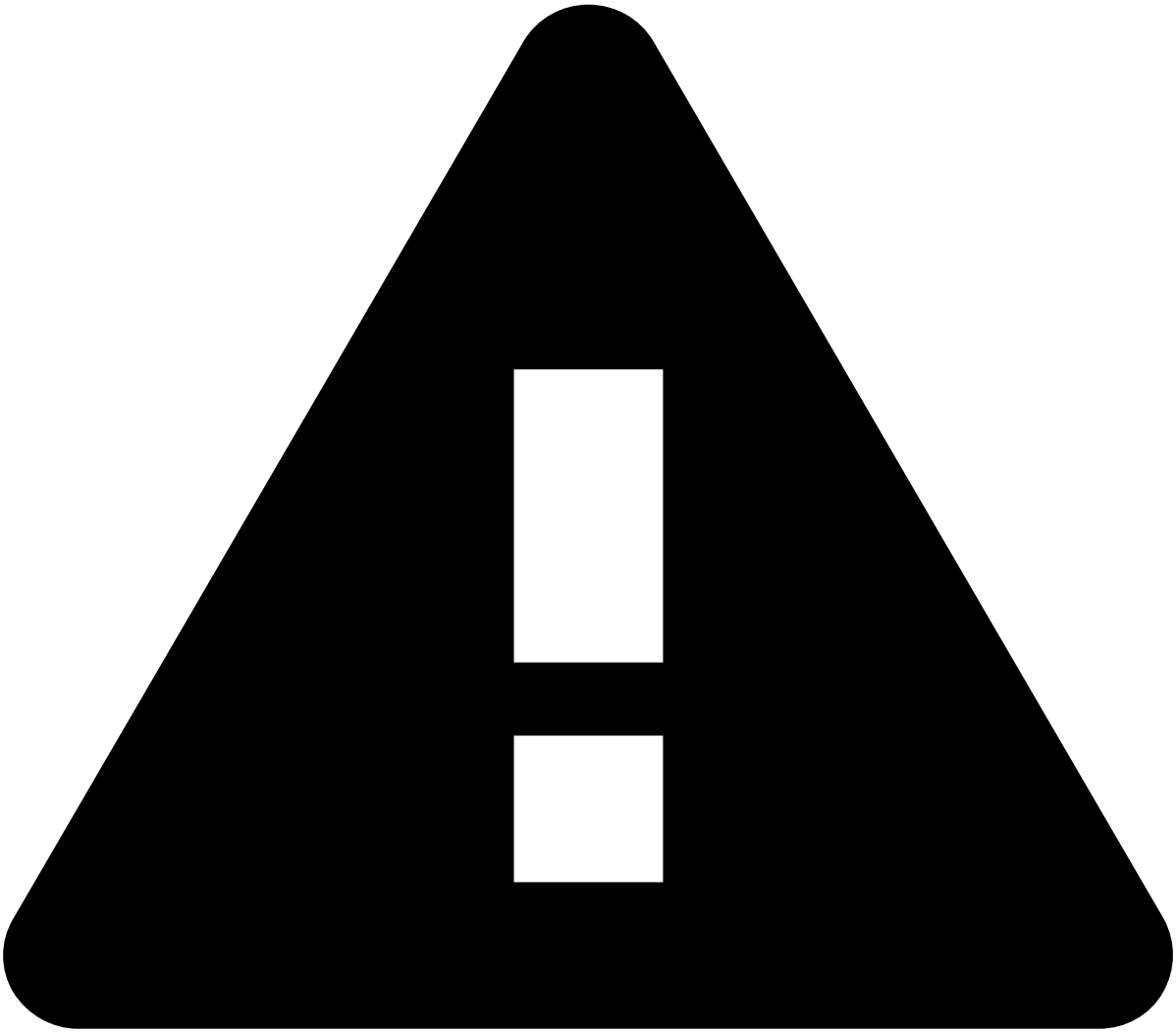


Mapping of data types from Parquet filesâ□□

Parquet files encode the type of each field, in addition to the values. When Parquet files are imported into Pulse, those types are automatically converted into the data types supported by Pulse, listed in the tables above. The mapping between Parquet data types and Pulse data types is as follows:

Parquet type	Pulse type	Configuration
`INT32`	Numeric	Integer
`INT64`	Numeric	Long
`INT64` with `TIMESTAMP` logical type annotation	Date Time	Timestamp in milliseconds (Long)
`INT96` (Deprecated in Parquet, but used in Spark for timestamps)	Date Time	Timestamp in milliseconds (Long)
`FLOAT`	Numeric	Float
`DOUBLE`	Numeric	Double
`BINARY` with `STRING` logical type annotation	String	String
`BINARY` with `ENUM` logical type annotation	String	String
`BINARY` with `JSON` logical type annotation	String	String
`BINARY` with `NULL` logical type annotation	String	String
`BOOLEAN`	Categorical	Boolean

Parquet type	Pulse type	Configuration
`MAP`	Complex Field	Map
`MAP_KEY_VALUE`	Complex Field	Map
`LIST`	Complex Field	List
Nested Object	Complex Field	Tuple



caution

Not all data types supported by the Parquet format can be imported into Pulse. Fields whose types are not mapped to a Pulse data type as described in the table above will be ignored.

Impact of data types on memory consumptionâ□□

Adjusting the data types can significantly reduce the memory footprint of individual records. This can be extremely important in an application that handles thousands of events per second. The application will probably define multiple time windows that store events in memory.

You must be aware about the size occupied by your events, to be sure that there is enough memory for the events in the time windows you eventually define.

As an example, consider an application that processes 300 events per second and whose schema has 10 string fields that store 8 characters each. Defining a 24 hours window will result in the following memory consumption:

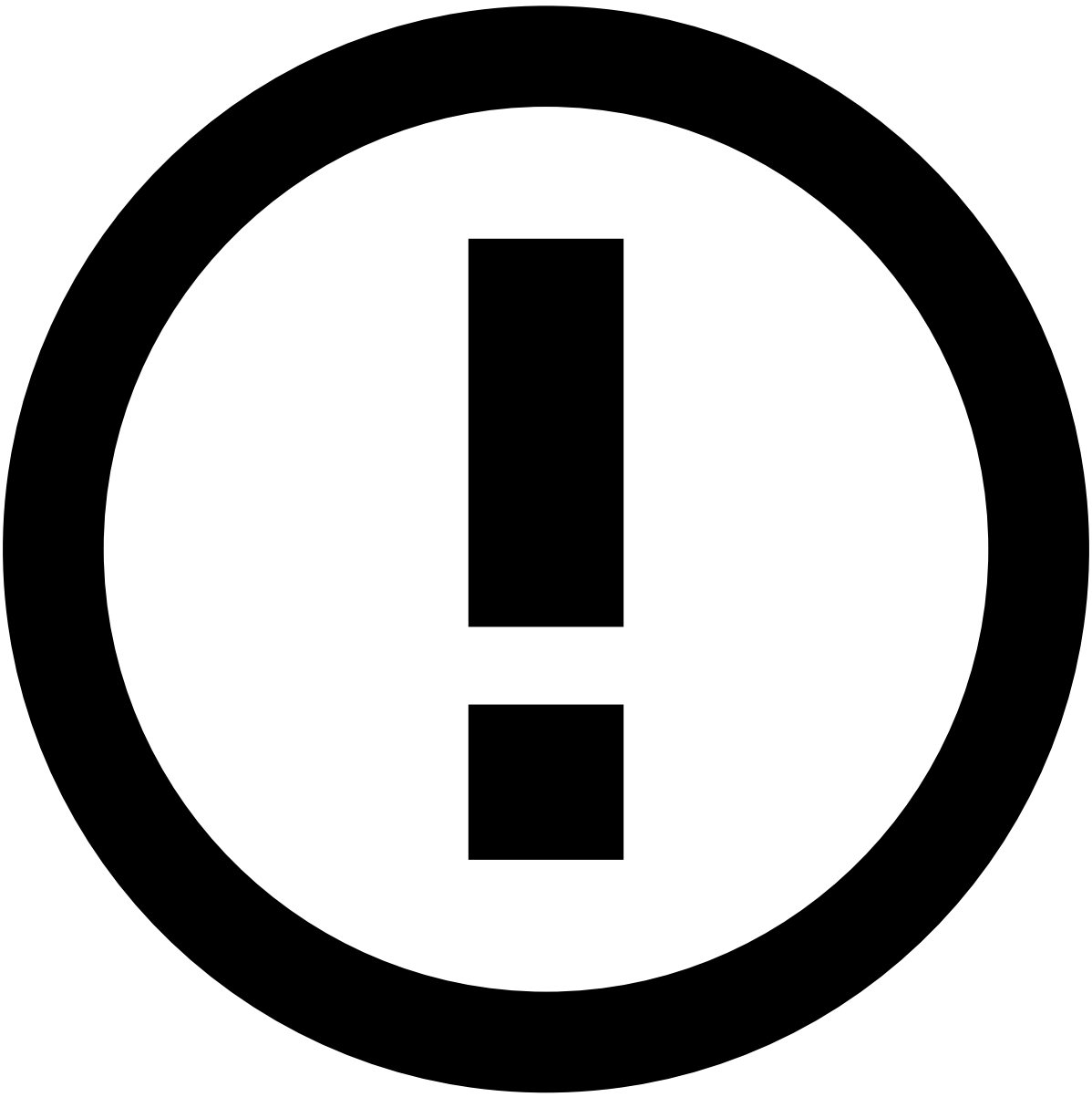
24 hours Ã□**60** minutes Ã□**60** seconds Ã□**300** events Ã□**10** fields Ã□**88** bytes each = **21 GB**

If you replace all string types with integers you will end up with:

24 hours Ã□**60** minutes Ã□**60** seconds Ã□**300** events Ã□**10** fields Ã□**16** bytes = **4 GB**

This alternative is much lighter than 21 GB, in fact it is about **5 times** smaller.

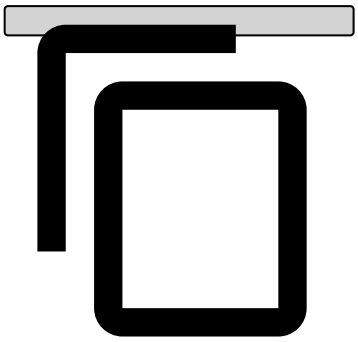
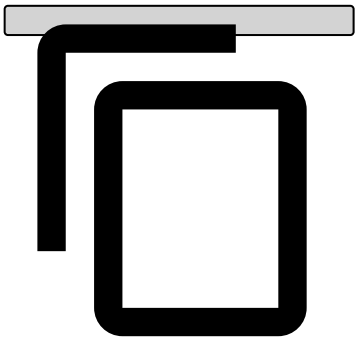
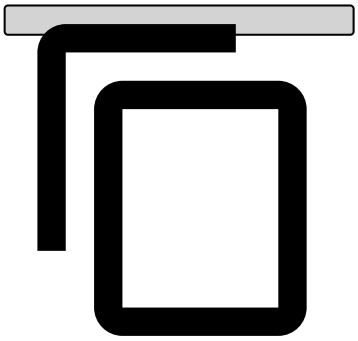
Try to choose the appropriate field types whenever possible to help reduce the memory used.

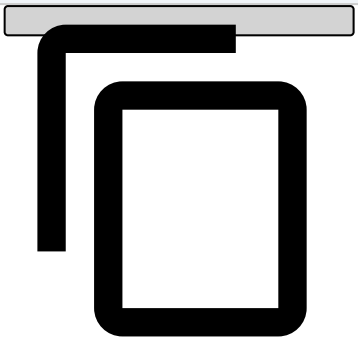
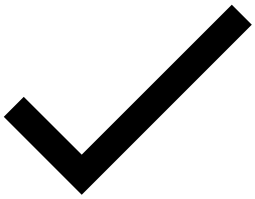
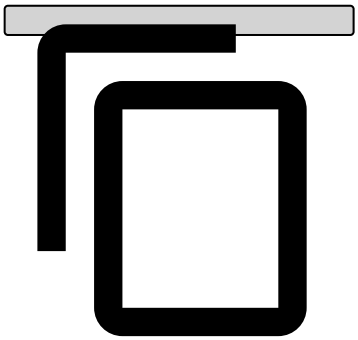
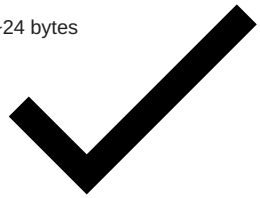
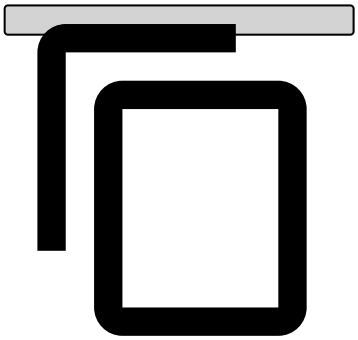
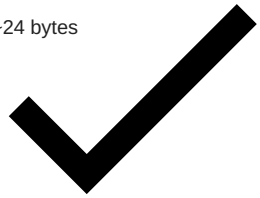
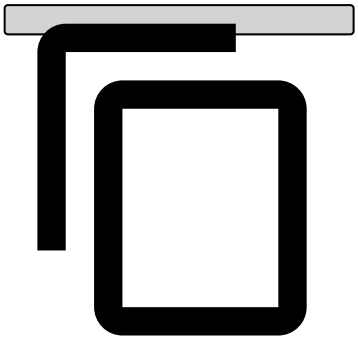
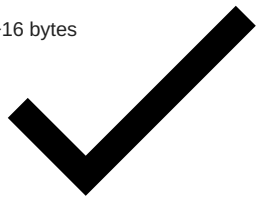


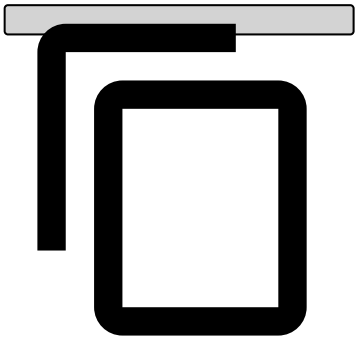
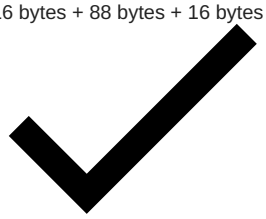
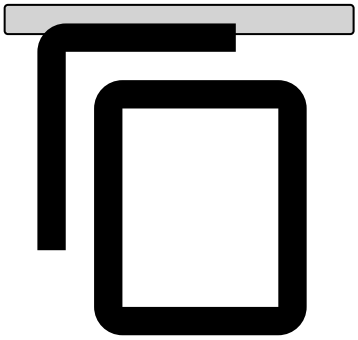
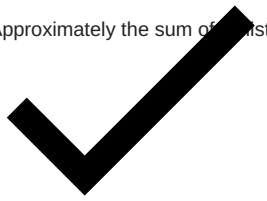
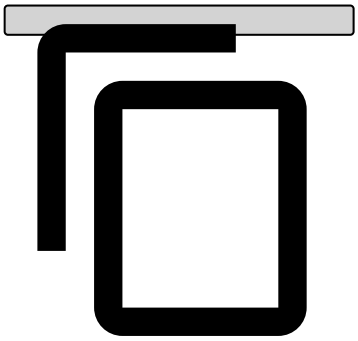
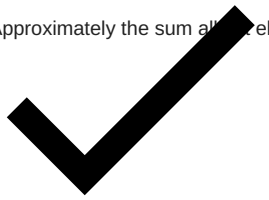
info

Since Strings use quite a lot of memory, only use them to save text that can't be categorized. Otherwise, there are other field types that suit data better.

The following table contains the size of each available data type:

Type	Example	Size (64bits JVM)
String	`"New York"` 	~2 bytes * number of characters + 68 bytes + padding needed to be a multiple of 8. (The example would occupy 88 bytes)
UTF8 String	`"New York"` 	~1 byte * number of characters + 24 bytes + padding needed to be a multiple of 8. (The example would occupy 36 bytes)
Integer	`1` 	~16 bytes
Float	`1.0`	~16 bytes

Type	Example	Size (64bits JVM)
		
Double	`1.0` 	~24 bytes 
Long	`1000000000` 	~24 bytes 
Boolean	`true` 	~16 bytes 

Type	Example	Size (64bits JVM)
Tuple	`{ amount: 2, country: "New York", timestamp: 123456 }` 	Approximately the sum of all tuple fields' sizes. In the example's case: 16 bytes + 88 bytes + 16 bytes + Tuple object overhead, so about 120 bytes. 
List	`[1, 2, 44, 1]` 	Approximately the sum of [illegible]st elements' sizes. 
Set	`["Alaska", "Arizona"]` 	Approximately the sum al[illegible] elements' sizes. 
Map	`[1: "Alaska", 2: "Minnesota"]`	Approximately the sum of all map elements' sizes.

Type	Example	Size (64bits JVM)